

Universidad de Antioquia
Facultad de Ingeniería
Informática II, 2026-2
Informe preliminar

10 de septiembre de 2026

Nombre: Luis Miguel Cohen Teherán.

Desafío I: Sweet Crush con bits

1. Contextualización.

El objetivo de este desafío es implementar una versión por consola del juego Sweet Crush, en C++ y sobre un proyecto Qt, con programación estructurada. El tablero es rectangular, de F filas y C columnas (valores que se leen al iniciar), y tiene seis tipos de ficha. Cada posición se representa con exactamente 3 bits dentro de un bloque continuo de unsigned char reservado con new. No se usa un byte ni un entero por ficha, ni se desempaqueta el tablero de forma permanente a una matriz auxiliar. El jugador no intercambia fichas. Las opciones del menú son atómicas, es decir, en cada turno se elimina una coordenada, se agrega o elimina una fila, se agrega o elimina una columna, o se sale. El programa no cierra la partida por su cuenta.

Después de eliminar una ficha, las que están encima descienden por la misma columna y las posiciones libres se completan con fichas nuevas. Tres o más fichas iguales consecutivas, en horizontal o en vertical, forman una combinación y se eliminan; una misma ficha puede participar en las dos direcciones. Si al reorganizar el tablero aparecen combinaciones nuevas, el proceso se repite y ese ciclo es lo que se llama cascada. Al insertar una fila o una columna se generan fichas aleatorias. Eso no se cuenta como cascada, porque en clase se definió la cascada como la caída vertical después de una eliminación automática. Sí se revisan combinaciones después de insertar, después de cambiar el tamaño o de meter fichas nuevas. Si salen grupos de tres o más, se sigue el ciclo normal de borrar, caer y rellenar, y esas caídas sí cuentan como cascadas. Al eliminar una fila o una columna se cierra el espacio y cambian las dimensiones reales. Luego de cada jugada se muestran dos vistas, sin borrar la consola, una es la trama en binario (el bloque de bits en memoria) y la otra es el tablero con símbolos, con filas y columnas numeradas. Con ellas se reportan las dimensiones, las eliminaciones del usuario, el total de fichas eliminadas, las combinaciones, las cascadas de esa jugada y el puntaje. Cada combinación eliminada suma un punto.

2. Análisis.

Con 3 bits hay ocho combinaciones. Se asignan 000, 001, 010, 011, 100 y 101 a las seis fichas. El código 110 queda como vacío transitorio, solo mientras dura una caída o el borrado de una combinación, y el 111 no se usa en esta versión. En consola cada tipo se muestra con un carácter distinto, por ejemplo #, @, O, X, + y %. El tablero se recorre por filas, de modo que el índice lineal es $\text{fila} \times C + \text{columna}$ y el bit de arranque es $\text{índice} \times 3$. La información válida comienza en el LSB (Least Significant Bit) del byte 0. Si $3 \times F \times C$ no es múltiplo de 8, los bits que no representan ficha se agrupan a la izquierda de toda la trama, en los bits de mayor peso del último byte, y no pueden quedar en el medio. Una ficha puede caber en un solo byte o partirse entre dos. Ambos casos se resuelven con máscaras y con los operadores << y >>. Antes de desplazar hay que conservar los bits que se quieren mantener, porque el corrimiento no es reversible.

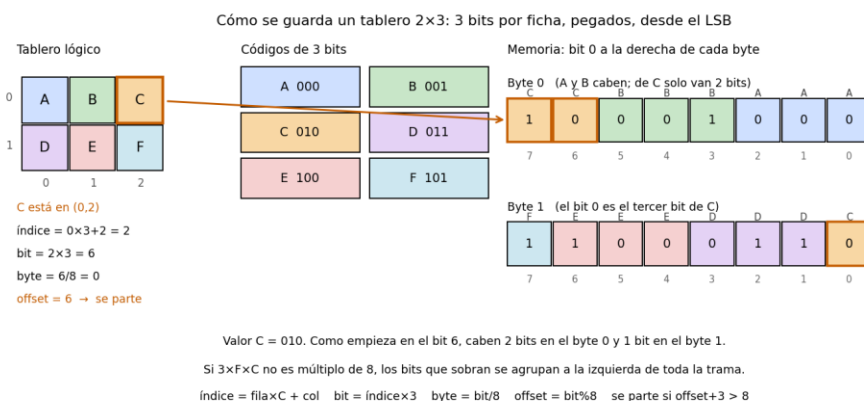


Figura 1. Empaquetado de un tablero 2x3 desde el LSB. La ficha C se parte entre el byte 0 (bits 6 y 7) y el bit 0 del byte 1.

El tamaño mínimo del bloque es $(3 \times F \times C + 7) / 8$ bytes, con división entera. La primera reserva usará exactamente ese valor, esto no deja capacidad extra ni un factor de crecimiento. Por eso, agregar una fila o una columna obliga a reservar un bloque mayor; como máximo sobran 7 bits, nunca un byte completo libre. Al eliminar, las dimensiones lógicas se actualizan de inmediato, pero la memoria física se compacta sobre el mismo arreglo. La ocupación se calcula como $(F \times C \text{ actuales})$ dividido por las celdas de la última reserva física. Solo si esa razón baja de 0.65 se reserva un bloque más pequeño y se libera el anterior. En un tablero 10x10 hay 100 celdas reservadas; al quitar una fila quedan 90, la ocupación es 0.90 y el puntero no cambia.

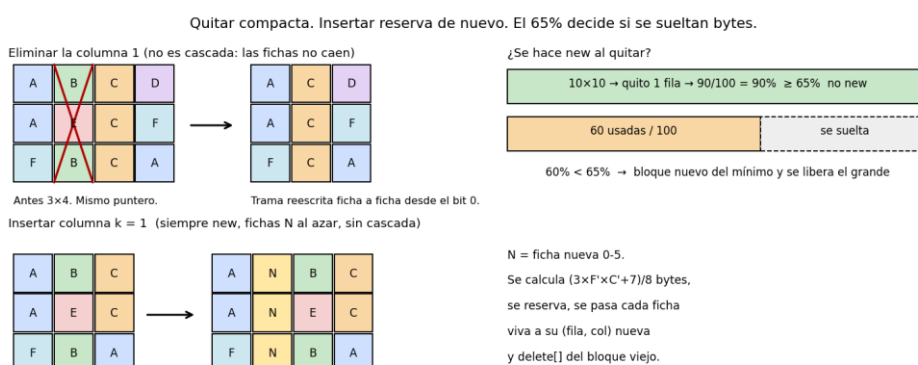


Figura 2. Al quitar una columna se compacta la trama, sin caída. El 65% determina si se libera memoria. Insertar una fila o una columna siempre reserva un bloque nuevo.

Hay que distinguir tres tipos de eliminación, y no se mezclan entre sí.

- La del usuario sobre una ficha es una opción de menú y se cuenta como eliminación del usuario.
- La de combinaciones es automática, o sea tres o más fichas iguales consecutivas. La de una fila o una columna también es de menú y es la única que modifica las dimensiones.
- Quitar una columna no es una cascada, porque solo se compactan los espacios. Una cascada es una caída vertical después de una eliminación automática, aunque no genere otra combinación, y aun así se cuenta.

Si hay varias combinaciones al mismo tiempo, se marcan y se eliminan todas, después se aplica la caída y el relleno, y después se vuelve a buscar. Una forma en T, un tramo horizontal y uno vertical que comparten ficha, cuenta como una sola combinación, no como dos. Dos secuencias que no comparten ficha cuentan como dos. No se imprimen los tableros intermedios.

3. Esquema de tareas y diseño

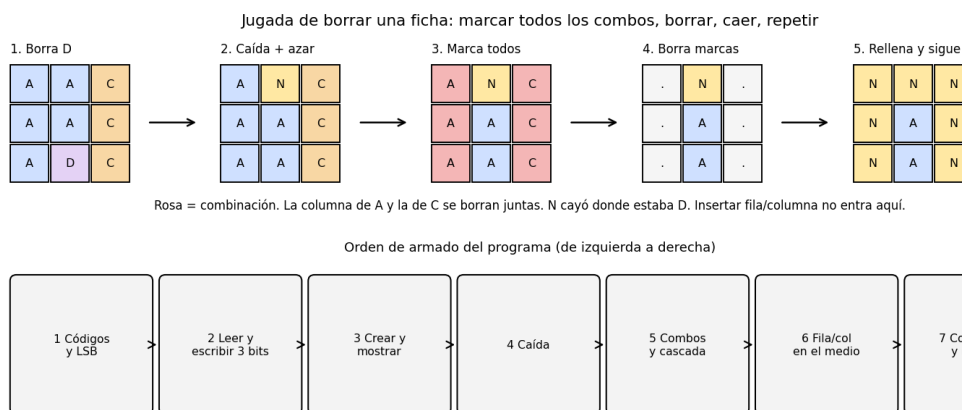


Figura 3. Flujo de una jugada, donde se marcan todas las combinaciones y después se aplica la caída. Debajo aparece un orden posible para ir construyendo el programa.

Por ahora el tablero se guarda con un puntero unsigned char* al bloque, más F, C, los bytes reservados y las celdas que había en la última reserva física. También hay que llevar los cinco indicadores de cada jugada y el puntaje. La regla de puntaje es una sola. Cada combinación suma un punto. El total de fichas eliminadas es otro contador, aparte. Si un mismo grupo quita cinco fichas, eso suma 1 al puntaje y 5 al total de fichas, no dos puntos. No puntuaría la ficha individual del usuario ni recortar una fila o una columna.

Para leer una posición se partirá del bit de arranque, del byte y del desplazamiento dentro de ese byte. Si los 3 bits caben ahí, se extraen con una máscara; si no caben, se arman con lo que queda de ese byte y con el siguiente, desde el LSB. Escribir sería el proceso inverso. La idea es que la caída, las combinaciones y el cambio de tamaño usen ese mismo acceso, para no terminar con dos formas de empaquetar. El detalle de las máscaras se irá ajustando al implementar.

Al crear el tablero se reserva el mínimo de bytes y cada celda se llena al azar entre los seis tipos, con distribución uniforme. Hay que mostrar la trama en binario, del bit 7 al 0 de cada byte, y el tablero con símbolos y con filas y columnas numeradas, sin borrar la consola. Si el usuario da una coordenada válida, esa posición quedaría vacía y la columna bajaría, completando arriba con fichas nuevas. Esa caída solo sumaría cascada cuando venga de una combinación automática, no cuando se quite una columna desde el menú.

Las combinaciones se buscarían por filas y por columnas, de tres o más iguales. Si una ficha entra en cruz, se procesaría una sola vez. Lo mencionado en clase es borrar primero todas las marcas, después caer y rellenar todo el tablero, y después volver a buscar. Insertar en una posición intermedia movería lo que queda y llenaría la franja nueva al azar, con un bloque del tamaño justo. Después se buscan combinaciones. El insertar no se cuenta como cascada. Eliminar compactaría en el mismo arreglo, y solo si la ocupación baja de 0.65 se reduciría la reserva. Cómo desplazar esos tramos de bits sin desalinearse es

parte de lo que hay que resolver en la implementación. Si no quedan filas, el menú dejaría de ofrecer eliminar fila, e igual con las columnas.

Se trabajará en varios archivos .h y .cpp, uno para la trama, otro del tablero y otro de la partida, aunque esto durante el desarrollo del Proyecto podría cambiar. El archivo principal quedaría para crear el tablero y atender el menú. El azar se trabajará con `cstdlib` y `ctime`. Se llama a `srand` una sola vez al iniciar el programa y después se usa `rand` para obtener un número entre 0 y 5, con distribución uniforme entre los seis tipos de ficha. Esa lógica queda en una sola rutina, y la usan la creación del tablero, el relleno al caer y la inserción de filas o columnas.